Azure implementation

Which Azure resources, identities, and lifecycle rules run the reference deployment, and what changes for production.

CONTENTS

1. Deployed topology
2. Identity and access
3. Deployment and lifecycle
4. Observability
5. Production hardening

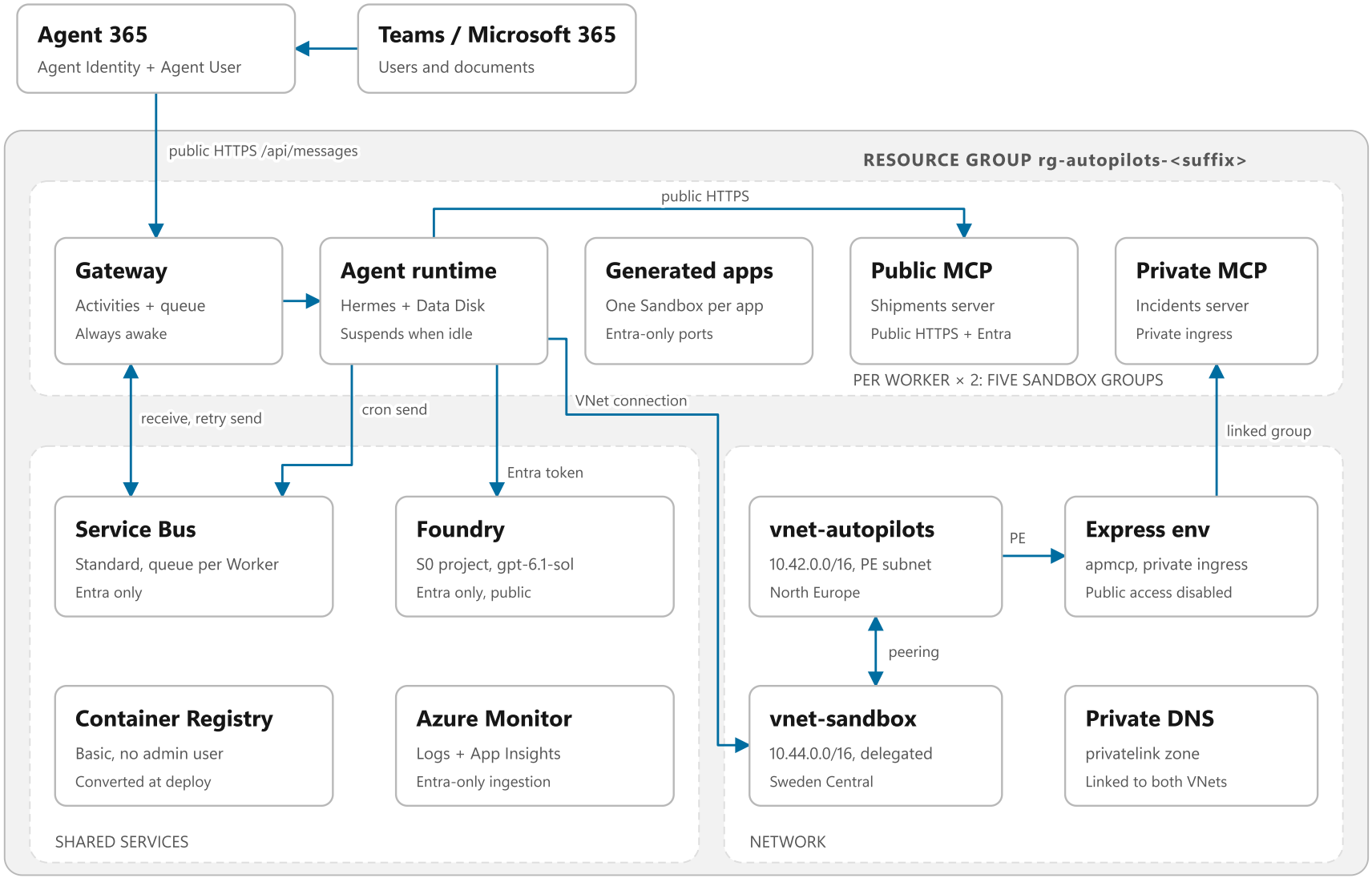
DEPLOYED TOPOLOGY

Each Worker gets five Sandbox Groups; shared services stay in one resource group.

The reference deployment runs two Workers, `hermes` and `hermes2`, in one resource group. Sandbox Groups, Foundry, and monitoring run in Sweden Central; Service Bus, Container Registry, and the workload VNet run in North Europe.

Solid: connection in use Dashed: optional connection or component

01 / Deployed Azure topology



Service Bus, Foundry, Container Registry, and Azure Monitor use Entra authentication on public endpoints. Only the private MCP path is private.

SOURCE

Platform layer · Worker layer · Networking

IDENTITY AND ACCESS

01

Grant each identity only the roles its service uses.

Terraform assigns every Azure role; no keys or registry tokens exist.

Each Sandbox Group has its own user-assigned managed identity `id-<worker>-<role>-*`. Every data-plane service has local authentication disabled, so these assignments are the complete Azure access model.

PRINCIPAL	ROLE	SCOPE
Deployer	Sandbox Data Owner	All five Sandbox Groups of the Worker
Gateway managed identity	Sandbox Data Owner	Generated-apps Group only
Runtime managed identity	Cognitive Services OpenAI User	Foundry account
Runtime, gateway, private-MCP, and public-MCP identities	AcrPull	Container Registry
Gateway managed identity	Service Bus Data Receiver and Data Sender	Worker queue, when scheduling or document retry is enabled
Agent Identity	Service Bus Data Sender	Worker queue
Gateway and runtime identities	Monitoring Metrics Publisher	Application Insights
Foundry project and operator	Log Analytics Reader, Monitoring Reader	Monitoring resources

SOURCE

Worker role assignments · Monitoring roles

02

Agent 365 objects come from scripts, not Terraform.

Blueprint, Agent Identity, and Agent User are Entra objects managed outside Terraform.

1

Create identities

`worker_lifecycle bootstrap` creates the agent blueprint, Agent Identity, and licensed Agent User, and requests the delegated Microsoft 365 permissions.

2

Federate the managed identities

`setup_identity` writes blueprint federated credentials for the gateway and runtime managed identities. It runs under a login that directly owns the blueprint.

3

Register the endpoint

`setup_agent365` sets the messaging endpoint to the gateway URL plus `/api/messages` and publishes the Agent User package.

SOURCE

Deployment · Agent 365 setup · Token exchange

DEPLOYMENT AND LIFECYCLE

03

Two Terraform layers; one workspace per Worker.

Infrastructure, images, and Sandboxes are separate, explicit steps.

PLATFORM

Resource group, registry, Service Bus namespace, Foundry, monitoring,

APPS

Per-Worker identities, Sandbox Groups, VNet connections, queue,

VNets, Express environment, private endpoint, and DNS.

and role assignments. Workspaces `autopilot-hermes` and `autopilot-hermes2`.

Images are built and converted between the two applies, and the deployment script then creates the Sandboxes. `hermes2` enables user scheduling and the Service Bus Dream; `hermes` leaves them off.

> SOURCE
>
> Deployment commands · Service deployment · Runtime deployment

04

Registry conversion belongs to deployment.

Running workloads receive prepared disk-image IDs, not registry credentials.

Terraform
Creates infrastructure, identities, networking, and configuration.

Build and convert
Pins dependencies and converts each service image once.

Deploy services
Uses disk-image IDs, then records healthy endpoints.

ACR conversion runs through the native-v2 disk-image API and authenticates with each Sandbox Group's managed identity, which holds `AcrPull`. No registry token or admin credential exists. Reusing a prepared image skips conversion.

> SOURCE
>
> Conversion identity decision · Conversion implementation · Commands and owner preflight

05

Only the gateway stays awake.

Suspension saves cost; image changes create new Sandboxes.

SERVICE	IDLE BEHAVIOR	ON IMAGE CHANGE
Gateway	Never suspends	New Sandbox and URL; update the Agent 365 endpoint
Agent runtime	Suspends to Disk after 300 s; self-ingress protects active work	New Sandbox on the same Data Disk
Public and private MCP	Suspend to Disk after 1,800 s; OnDemand ingress wakes them	New Sandbox and URL, recorded by the capture
Generated apps	Suspend after 5 minutes	Created on demand; expire after 24 hours by default

A failed Sandbox is replaced by a healthy one before the old one is deleted. An environment-only change updates a Sandbox in place.

> SOURCE
>
> Service lifecycle · Generated apps

OBSERVABILITY

06

Correlate application work without exporting its content.

W3C context crosses the bridge and wrapper; native callbacks expose model and tool execution.

Application Insights receives metadata-only OpenTelemetry spans. Worker and session attribution connect the gateway request, runtime invocation, model calls, and tools. Prompts, responses, bearer tokens, and tool payloads do not belong in this trace stream.

The Foundry external-agent registration links this execution to its monitoring surface. Registration does not move the agent into Foundry hosting or establish evaluation quality.

<details>
<summary>Inspect correlation limits</summary>

The Sandbox-origin egress path preserves TraceId but can rewrite ParentId; its intermediate platform span is not available in application telemetry. Correlation is therefore useful without a complete platform-hop tree. The native `/v1/responses` path also lacks explicit session correlation.

An interrupted native handoff retains its lease to prevent misattributing later spans. Restart the affected wrapper/native runtime processes before reuse; do not fabricate a parent span or silently clear the fence.
</details>

SOURCE

Telemetry policy · Native handoff · Foundry registration

PRODUCTION HARDENING

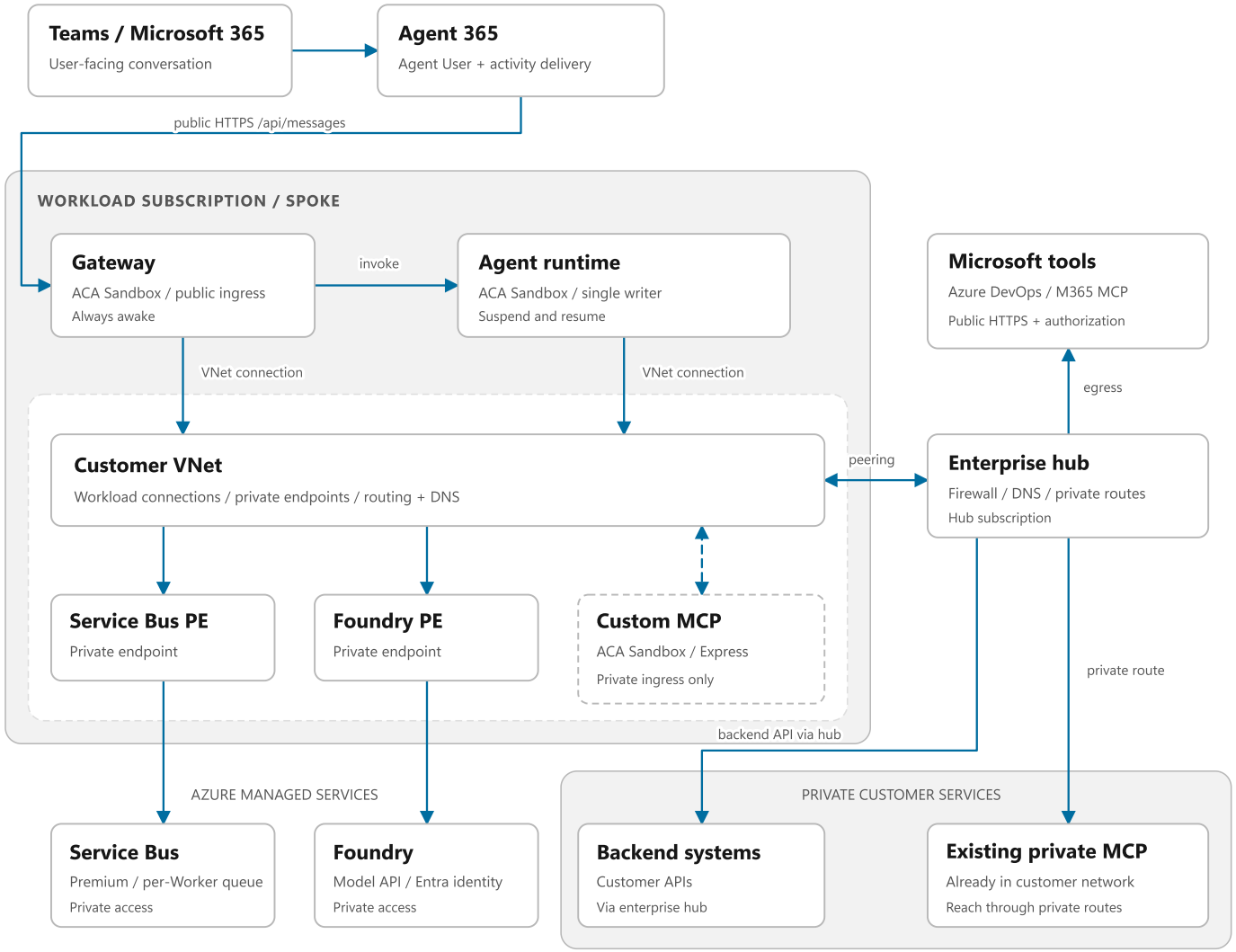
07

Production adds private endpoints and an enterprise hub.

Network changes limit reachability; they do not replace identity checks.

This is a target design, not the deployed state. Service Bus moves to Premium, and Service Bus and Foundry get private endpoints in the workload VNet. A peered enterprise hub provides DNS, firewall, controlled egress, and routes to existing private MCP services and backend APIs.

02 / Production target network



PE means private endpoint. Custom MCP reaches backend systems through the workload VNet, peering, and enterprise hub before entering the private customer network.

Tools fit three patterns: Microsoft-hosted MCP services on public HTTPS, existing MCP services in the customer network, or a custom MCP adapter deployed with the solution. Identity checks stay the same in every pattern.

Terraform owns the infrastructure and every Azure role, scripts own the Agent 365 identities and images, and each Worker keeps its own Sandbox Groups and queue.

COMPONENT REFERENCE

▼ Agent 365

Entra and Microsoft 365 objects

Each Worker has an agent blueprint, an Agent Identity, and a licensed Agent User. Scripts create them and register the messaging endpoint as the gateway URL plus `/api/messages`. Terraform does not manage these objects.

▼ Teams and Microsoft 365

User-facing workloads

Users mention the Agent User in Teams, mail it, or comment in Office documents. Work IQ and Graph calls from the runtime run as the Agent User.

▼ Gateway Sandbox Group

Auto-suspend disabled

Python bridge with public ingress. It validates Activity Protocol requests, calls the runtime URL, receives Service Bus messages, and creates generated-app Sandboxes. Its managed identity has Sandbox Data Owner only on the generated-apps Group. It also has a VNet connection.

▼ Agent runtime Sandbox Group

Suspends to Disk after 300 s idle

Hermes with the wrapper and loopback MCP adapters. Native execution and runtime request scopes send self-ingress while work runs; the port is OnDemand. The Worker profile lives on a Data Disk; resume keeps the same Sandbox ID. Its VNet connection reaches the private MCP.

▼ Generated apps Sandbox Group

One Sandbox per app

The gateway starts each Worker-created app in its own Sandbox with Entra-protected ports. Apps suspend after 5 idle minutes and expire after a lifetime of 24 hours by default (15 minutes to 7 days).

▼ Public shipments MCP

Public HTTPS

Demo MCP server on a public Sandbox port. It validates the Agent Identity token for tenant, issuer, audience, caller, and role, and suspends after 1,800 s idle.

▼ Private incidents MCP

Private ingress

Demo MCP server whose Sandbox Group is linked to the Express environment. The link is irreversible. It validates the same Entra claims as the public server.

▼ Service Bus namespace

Standard, local auth disabled

Namespace `apschedule-*` in North Europe, TLS 1.2, public endpoint. One queue `worker-<name>` per Worker: lock 5 minutes, message TTL 14 days, duplicate detection 1 hour.

▼ Foundry

AIServices S0, local auth disabled

Project `autopilots-project` with deployment `gpt-6-1-sol` (model `gpt-6.1-sol`, GlobalStandard, capacity 100) in Sweden Central. The runtime managed identity has Cognitive Services OpenAI User. Foundry also holds the external-agent registration and an Application Insights connection.

▼ Container Registry

Basic, admin user disabled

Images are built and pushed, then converted once to Sandbox disk images by the deployment script. Conversion authenticates with each Sandbox Group's managed identity, which holds AcrPull (ADR 0021).

▼ Azure Monitor

Log Analytics + Application Insights

Both have local authentication disabled. Gateway and runtime managed identities publish metadata-only OpenTelemetry spans with Monitoring Metrics Publisher.

▼ Workload VNet

10.42.0.0/16, North Europe

Holds subnet `snet-private-endpoints` (10.42.2.0/24) with the private endpoint `pe-apmcp-*` for the Express environment.

▼ Express managed environment

Public network access disabled

Environment `apmcp-*` in Sweden Central supplies private ingress for the linked private-MCP Sandbox Group.

▼ Sandbox VNet

10.44.0.0/16, Sweden Central

Subnet `snet-sandbox` (10.44.0.0/24) is delegated to `Microsoft.App/environments`. Runtime and gateway Groups attach through VNet connections. Peered in both directions with the workload VNet.

▼ Private DNS zone

privatelink.swedencentral.azurecontainerapps.io

Resolves the private MCP hostname to the private endpoint. Linked to both VNets.

▼ Teams and Microsoft 365 entry points

Reference implementation

Personal messages and explicit Agent User mentions are supported entry points. Delivered replies, selected reactions, Mail notifications, and Office comment events have workload-specific handling.

Shared groups are a shared audience. Private per-person delivery and human on-behalf-of identity are not implemented.

▼ Agent 365 channel and identity binding

Channel identity

Each Worker has an Agent Identity and Agent User. Agent 365 binds its endpoint and permissions, then delivers Microsoft 365 activities to `/api/messages`. The Agents SDK authenticates incoming requests.

▼ Gateway Sandbox

Always awake

The gateway authenticates activities, binds conversations, wakes the agent runtime, consumes Service Bus, and delivers replies. It stays awake for queued and post-acknowledgement work. Model calls and MCP traffic go directly from the runtime.

▼ Agent runtime

ACA Sandbox

Deployment creates agent compute; gateway requests wake it through OnDemand ingress. State survives on a separate Data Disk. One runtime writes the Worker profile at a time. Keepalive protects active execution; five idle minutes suspend compute to Disk. OnDemand ingress resumes the same Sandbox ID.

▼ Customer VNet

Workload connectivity

Gateway and runtime attach here. Private endpoints connect managed services; private MCP ingress joins the same network boundary. Peering reaches the enterprise hub. Routes and DNS provide reachability; identity still controls access.

▼ Service Bus private endpoint

Target design

A private endpoint and DNS connect the workload VNet to Service Bus Premium. Clients still authenticate with Entra identities.

▼ Foundry private endpoint

Target design

The Foundry private endpoint connects model access to the workload VNet. Private DNS resolves the model endpoint to its private address; the runtime still authenticates with its managed identity.

▼ Custom MCP

Optional adapter

Add an ACA-hosted adapter when a backend lacks MCP. It exposes a private MCP interface and reaches backend APIs through the workload VNet and enterprise hub. Use a supported private-ingress hosting configuration.

▼ Service Bus queue and dead-letter handling

Target design: Premium / private endpoint

The per-Worker queue carries cron, Dreaming, and document-retry triggers. The awake gateway uses PeekLock and explicit settlement. Runtime cron sends as Agent Identity; gateway receive and retry send use managed identity.

▼ Foundry model inference

Target design: private endpoint

The agent runtime calls Foundry through the private endpoint. Runtime managed identity and RBAC authorize inference; private routing and DNS provide reachability. The model service remains outside the workload subnet.

▼ Enterprise hub

Shared network services

VNet peering connects the workload spoke to hub routing, DNS, and firewall services. The hub provides controlled public egress and private routes to existing customer services, including MCP endpoints.

▼ Microsoft-hosted MCP tools

Public HTTPS

Use approved Microsoft-hosted tools, such as Azure DevOps and Microsoft 365 MCP integrations. Configure permissions for each service. Microsoft 365 operations use the approved Agent User flow; other services require their own supported connection.

▼ Existing private MCP

Customer-provided service

Connect to an MCP server already running in the customer network through private routing and DNS. Reuse its approved authentication and permissions; no new MCP host is needed in this solution.

▼ Backend systems

Customer APIs

Custom MCP reaches these APIs through the workload VNet, peering, and enterprise hub into the private customer network. Backends keep their existing interfaces and permissions; they do not need to implement MCP.